

Tutorial - 8

In this tutorial, you will design, debug and execute a few OpenACC programs, and analyze the compiler outputs.

1. Compile and execute the following program. Analyze the output of the *pgcc* compiler. Run the program as *./a.out 10*. Analyze the output with *async* clause added to the first parallel region.

```

1 #include<stdio.h>
2 #include<stdlib.h>
3
4 int main(int argc, char* argv[])
5 {
6     int ngangs = 1;
7
8     if (argc == 2)
9         ngangs = atoi(argv[1]);
10    else
11    {
12        printf(" Run the program as ./a.out 10");
13        return 1;
14    }
15
16    printf("\n ngangs = %d", ngangs);
17
18    #pragma acc parallel num_gangs(ngangs)
19    {
20        printf(" Hello world \n");
21        printf(" Bye world \n");
22    }
23
24    printf("Host \n");
25
26    #pragma acc parallel num_gangs(ngangs/2)
27    printf("Second pragma \n");
28
29    return 0;
30 }

```

2. Find and fix error(s) in the following program fragment trying to find a number in an integer array. After the code is working, improve parallelism in the code.

```

1 #pragma acc parallel
2     for (int i = 0; i < N; i++)
3         if (numbers[i] == mynumber) break;
4
5     if (i != N) printf("Number not found \n");
6     else printf("Number found at location %d \n", i);

```

3. Test the following OpenACC directives using suitable sample programs:

```

1 #pragma acc serial
2     num_gangs, num_workers, vector_length
3     private, firstprivate, if

```

4. Complete the following programs we have seen in the lecture, compile them using the -Minfo flag and analyze the output for the 3 versions given below.

```
1 Version 1:
2 #pragma acc kernels
3 {
4     printf("One \n");
5     for (int i = 0 ; i < 10; ++i)
6         a[i] = i;
7     printf("Two: a[9] = %d \n", a[9]);
8     for (int i = 0; i < 10; ++i)
9         a[i]* = 2;
10    printf("Three: a[9] = %d \n", a[9]);
11 }
```

```
1 Version 2:
2 #pragma acc kernels
3 {
4     printf("One \n");
5     for (int i = 0 ; i < 10; ++i)
6     {
7         a[i] = i;
8         printf("first loop %d \n", i);
9     }
10    printf("Two: a[9] = %d \n", a[9]);
11    for (int i = 0; i < 10; ++i)
12    {
13        a[i]* = 2;
14        printf("second loop %d \n", i);
15    }
16    printf("Three: a[9] = %d \n", a[9]);
17 }
```

```
1 Version 3:
2 #pragma acc kernels
3 {
4     printf("One \n");
5     for (int i = 0 ; i < 10; ++i)
6     {
7         a[i] = i;
8         printf("first loop %d \n", i);
9     }
10    printf("Two \n");
11    for (int i = 0; i < 10; ++i)
12    {
13        a[i]* = 2;
14        printf("second loop %d \n", i);
15    }
16    printf("Three \n");
17 }
```